

ADVANCED RAG-ENABLED TRANSFORMER EMBEDDING SYSTEM FOR INTELLIGENT KNOWLEDGE DISCOVERY FROM EXCEL DATASETS

AI Discussions • Cybersecurity Awareness • RAG Project Development | February 2026 – March 2026

Internship Organization:

Bits N' Bytes Cybersecurity Education, USA

Prepared By: Devadharshani B

Trainer: Vidya Bharadwaj

Abstract— In modern organizations, large volumes of structured data are stored in spreadsheet formats such as Microsoft Excel. Although these datasets contain valuable insights, extracting meaningful knowledge from them often requires manual searching or complex database queries. These traditional methods are inefficient and require significant technical expertise.

This paper presents an intelligent knowledge discovery system that enables users to interact with Excel datasets using natural language queries. The proposed system integrates Retrieval-Augmented Generation (RAG) with transformer-based semantic embeddings to retrieve relevant dataset rows and generate context-aware responses using large language models.

The system architecture consists of a React-based frontend dashboard, a FastAPI backend server, and a vector retrieval engine using FAISS. When a dataset is uploaded, each row is converted into semantic embeddings using transformer models. These embeddings are stored in a vector database that enables efficient similarity search. When users submit natural language queries, the system retrieves relevant rows and generates grounded responses using the retrieved context.

The proposed system also provides explainable AI outputs, including similarity scores, confidence metrics, and retrieved evidence. Experimental results demonstrate that the system improves the accessibility of structured data and significantly reduces the time required to extract insights from Excel datasets.

Keywords— Retrieval-Augmented Generation, Transformer Embeddings, Vector Databases, FAISS, Natural Language Processing, Knowledge Discovery, Artificial Intelligence

I. INTRODUCTION

The exponential growth of data across industries has created significant challenges in managing and extracting meaningful insights from structured datasets. A large portion of organizational data is stored in spreadsheet formats such as Microsoft Excel. While these datasets provide valuable information, traditional analysis techniques require manual filtering, sorting, or complex queries.

For non-technical users, accessing information from large datasets becomes a difficult task. This limitation highlights the need for intelligent systems that allow users to query structured data using natural language.

Recent advancements in **Natural Language Processing (NLP)** and **Large Language Models (LLMs)** have made it possible to build systems capable of understanding and generating human language. One promising approach is **Retrieval-Augmented Generation (RAG)**, which combines information retrieval techniques with generative language models.

RAG systems retrieve relevant documents or data entries and provide them as context to language models, ensuring that generated responses are grounded in factual information.

This research presents a **full-stack AI system** that enables users to upload Excel datasets and interact with them using natural language queries. The system leverages transformer embeddings and vector databases to perform semantic search and generate accurate responses.

II. LITERATURE REVIEW / RELATED WORK

Several studies have explored the application of artificial intelligence in information retrieval and knowledge discovery.

Transformer-based models such as **BERT** and **Sentence Transformers** have significantly improved semantic understanding of text data. These models enable the conversion of textual information into vector embeddings that capture contextual meaning.

Vector databases such as **FAISS** have been widely used for efficient similarity search in high-dimensional embedding spaces. FAISS enables approximate nearest neighbor search, allowing systems to retrieve relevant data points quickly.

Retrieval-Augmented Generation has emerged as an effective approach to enhance language model responses by incorporating retrieved contextual information. By grounding responses in retrieved data, RAG systems reduce hallucination and improve factual accuracy.

Recent research has demonstrated the effectiveness of combining transformer embeddings with vector retrieval for applications such as document search, question answering, and recommendation systems.

However, limited research has focused on applying these techniques to structured spreadsheet datasets. The proposed system addresses this gap by enabling natural language querying of Excel datasets.

III. Problem Statement

Organizations often rely on Excel spreadsheets for storing operational, financial, and analytical data. Although spreadsheets provide flexibility in data storage, retrieving specific insights from large datasets remains challenging.

Traditional approaches require manual searching, filtering, or writing SQL queries. These methods are time-consuming and often inaccessible to non-technical users.

Therefore, there is a need for an intelligent system that allows users to query structured spreadsheet datasets using natural language while providing accurate and explainable results.

IV. Proposed System

The proposed system introduces an AI-driven knowledge discovery framework that integrates vector retrieval and language generation.

The system architecture consists of three primary components:

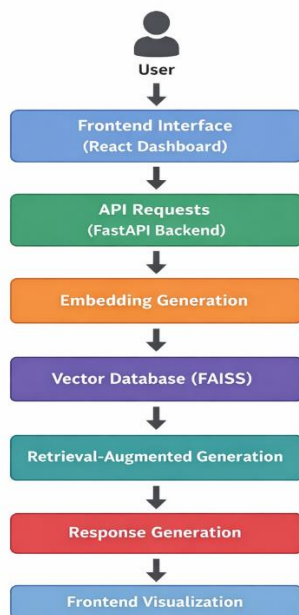
1. Frontend Interface
2. Backend Processing Engine
3. Vector Retrieval and Generation Pipeline

The frontend dashboard allows users to upload datasets and submit natural language queries. The backend processes datasets, generates embeddings, performs retrieval operations, and communicates with the language model to generate responses.

V. System Architecture

The architecture of the proposed system follows a layered design consisting of presentation, application, and data layers.

System Workflow:



VI. Methodology

A. Dataset Ingestion

The system accepts Excel files uploaded by users. The datasets are processed using Python-based data processing libraries.

Each row in the dataset is extracted and converted into a textual representation suitable for embedding generation.

Example:

Row Data:

Name: John

Department: HR

Salary: 50000

Converted Text:

"John works in the HR department and earns a salary of 50000."

B. Data Preprocessing

Data preprocessing ensures dataset quality and consistency. The following steps are performed:

- Removing duplicate rows
- Handling missing values

- Normalizing column formats
- Converting rows into textual chunks

C. Embedding Generation

Textual representations of dataset rows are converted into vector embeddings using transformer models.

The system uses Sentence Transformers (all-MiniLM-L6-v2) to generate 384-dimensional embeddings.

These embeddings capture semantic relationships between data entries.

D. Vector Database Storage

Generated embeddings are stored in a FAISS vector database.

FAISS enables efficient approximate nearest neighbor search, allowing the system to retrieve relevant dataset rows based on semantic similarity.

E. Query Processing

When a user submits a natural language query, the system converts the query into an embedding using the same transformer model.

The vector database then retrieves the most relevant dataset rows.

F. Retrieval-Augmented Generation

The retrieved dataset rows are provided as contextual information to a large language model.

The language model generates responses grounded in the retrieved context, ensuring accuracy and relevance.

VII. Implementation

The system was implemented using modern web development and AI frameworks.

Frontend:

- ✓ React (Vite)
- ✓ Tailwind CSS
- ✓ Axios
- ✓ Recharts

Backend:

- ✓ Python
- ✓ FastAPI
- ✓ PostgreSQL
- ✓ FAISS

AI Components:

- ✓ Sentence Transformers
- ✓ Vector similarity search
- ✓ Retrieval-Augmented Generation

VIII. Project Output

The system produces the following outputs:

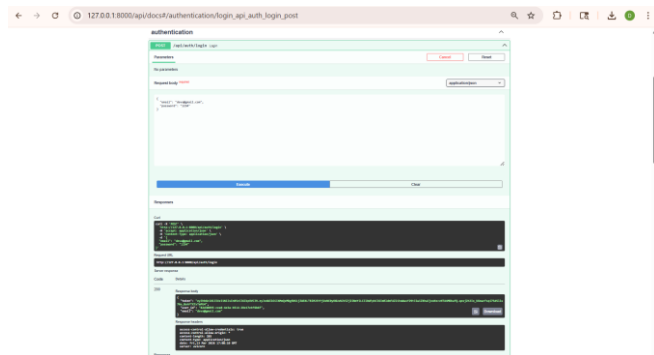
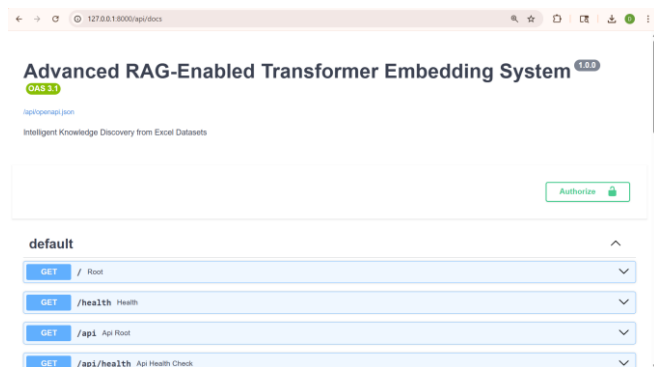
1. Natural language AI responses to user queries.
2. Retrieved dataset rows supporting the generated answer.
3. Similarity scores representing retrieval relevance.
4. Confidence scores indicating response reliability.
5. Performance metrics such as query latency.

Backend Screenshot:

1. Backend url

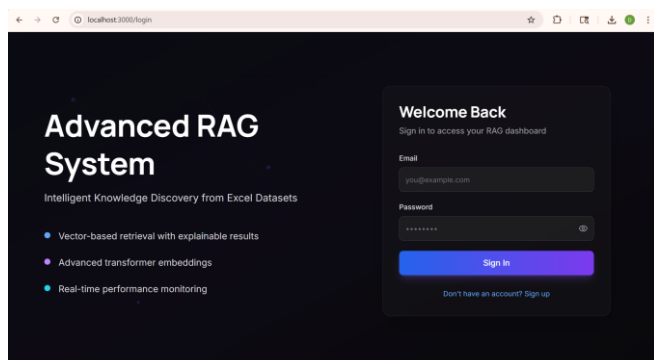
```
127.0.0.1:8000
{
  "message": "Advanced RAG-Enabled Transformer Embedding System",
  "version": "1.0.0",
  "status": "operational",
  "docs": "/api/docs"
}
```

2. Checking The API Works

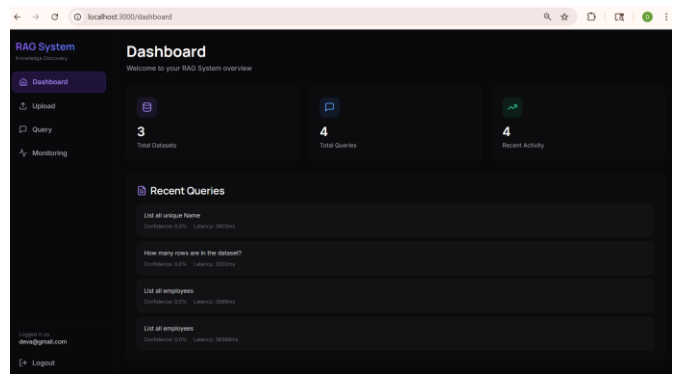


Frontend Screenshot:

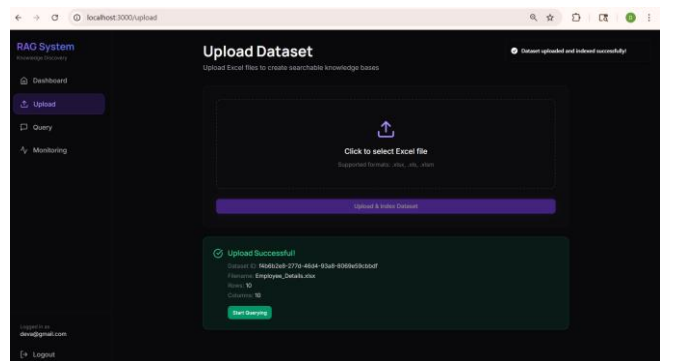
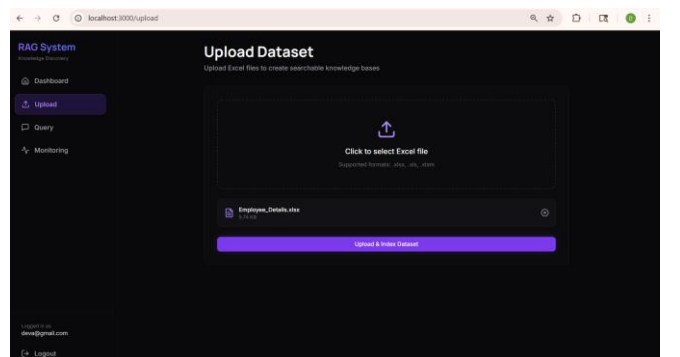
1. Login Page



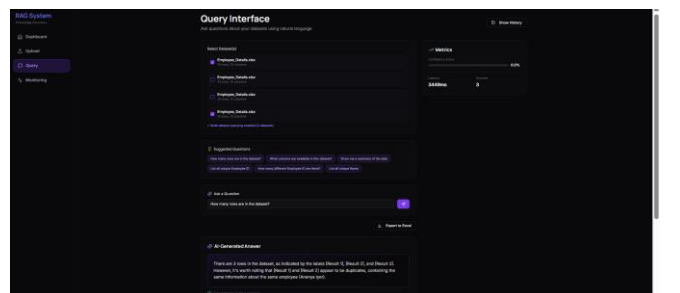
2. Dashboard



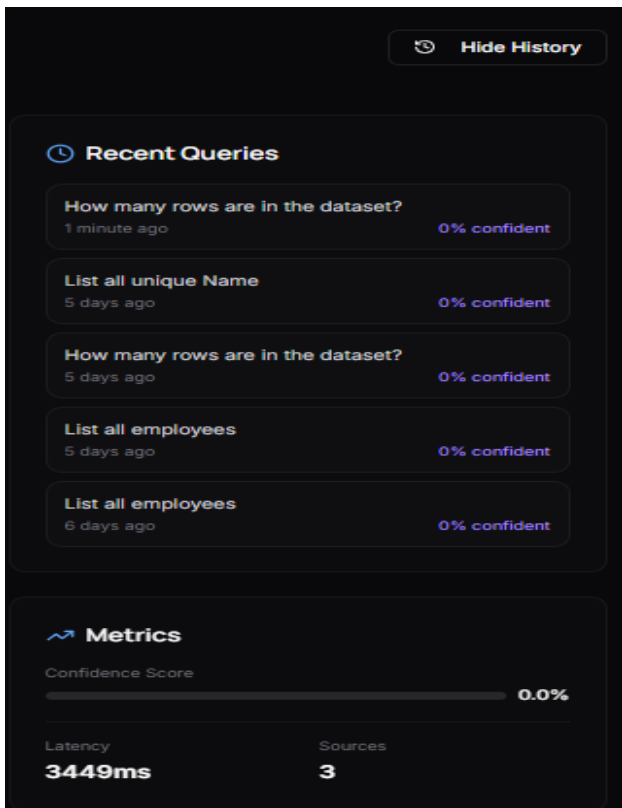
3. Upload Dataset



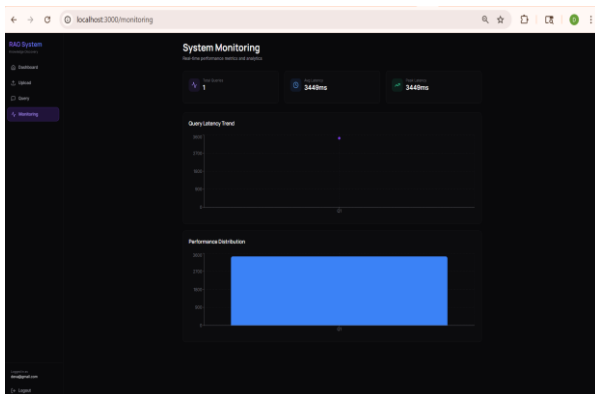
4. Query Interface



5. History



6. System Monitoring



IX. Challenges Faced and Solutions

During the development of the proposed system, several technical challenges were encountered related to dataset processing, retrieval accuracy, system integration, and deployment. The following subsections summarize the key challenges and the solutions implemented.

A. Large Dataset Processing

Handling large Excel datasets was one of the primary challenges. When large files were uploaded, generating embeddings for thousands of rows required significant computational resources and increased processing time.

Solution:

Batch processing techniques were implemented to generate

embeddings in smaller groups rather than processing the entire dataset at once. This reduced memory usage and improved processing efficiency.

B. Retrieval Accuracy

In the initial stages, the vector similarity search sometimes retrieved dataset rows that were not closely related to the user query.

Solution:

To improve accuracy, a hybrid retrieval strategy was introduced along with cross-encoder reranking. This approach helped select the most relevant rows for generating responses.

C. Frontend-Backend Integration

Integrating the React frontend with the FastAPI backend required proper handling of asynchronous API requests. Early implementations occasionally caused delays in receiving responses.

Solution:

The Axios library was used to manage API communication, along with proper error handling and asynchronous request management to ensure smooth interaction between the frontend and backend.

D. Deployment Challenges

Attempts were made to deploy the system using cloud platforms such as Render for the backend and Vercel for the frontend. However, several configuration and environment compatibility issues were encountered during deployment.

Solution:

Due to these challenges, the system was executed in a local development environment using Visual Studio Code. Running both frontend and backend locally allowed the complete functionality of the system to be demonstrated successfully.

E. System Performance Optimization

Optimizing system performance during embedding generation and vector retrieval was another challenge.

Solution:

Performance was improved by optimizing the embedding pipeline and implementing efficient indexing techniques within the FAISS vector database, which reduced query response time.

X. New Learnings During the Internship

This internship provided valuable experience in developing modern AI-powered applications.

Key learnings include:

- Understanding Retrieval-Augmented Generation pipelines
- Working with vector embeddings and similarity search
- Building scalable AI systems
- Full-stack development with modern frameworks
- Deploying applications on cloud platforms
- Implementing explainable AI systems

XI. Future Work

Future improvements may include:

- Integration with advanced vector databases such as Pinecone
- Support for multi-dataset analysis
- Real-time data ingestion
- Improved visualization dashboards
- Integration with enterprise data systems

XII. Conclusion

This project developed an AI-driven knowledge discovery system that enables users to interact with Excel datasets using natural language queries. By integrating transformer-based embeddings, vector similarity search, and retrieval-augmented generation (RAG), the system is able to retrieve relevant dataset rows and generate accurate and explainable responses.

The implemented system demonstrates how modern artificial intelligence techniques can simplify the process of analyzing structured data stored in spreadsheets. Instead of manually searching through large datasets, users can obtain insights quickly through a conversational interface.

Overall, the project highlights the potential of combining natural language processing, vector databases, and full-stack web development to create intelligent data exploration tools. Such systems can significantly improve the accessibility and efficiency of data analysis for both technical and non-technical users.

GitHub Repository

The complete implementation of this project is available on GitHub.

Repository Link:

<https://github.com/Devadharshani13/RAG-Enabled-Transformer-Embedding-System-for-Intelligent-Knowledge-Discovery-from-Excel-Datasets>

Recording Link:

<https://drive.google.com/drive/folders/162DS-9G093HZHb6BsHi2iREF2oodOkbz?usp=sharing>

ACKNOWLEDGMENT

As an intern, I would like to express sincere gratitude to **Bits N' Bytes Cybersecurity Education, USA**, for providing the opportunity to work on this internship project. Special thanks are extended to the trainer **Vidya Bharadwaj** for her guidance, support, and valuable insights throughout the project, which greatly contributed to its successful completion.

REFERENCES

- [1] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [2] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," *Proceedings*

of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP), 2019.

- [3] J. Johnson, M. Douze, and H. Jégou, "FAISS: A Library for Efficient Similarity Search and Clustering of Dense Vectors," *Facebook AI Research*, 2017.

- [4] T. Mikolov et al., "Efficient Estimation of Word Representations in Vector Space," *International Conference on Learning Representations (ICLR)*, 2013.

- [5] A. Vaswani et al., "Attention Is All You Need," *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

- [6] FastAPI Documentation, "FastAPI Framework for Building APIs with Python," Available: <https://fastapi.tiangolo.com/>

- [7] React Documentation, "React: A JavaScript Library for Building User Interfaces," Available: <https://react.dev/>

- [8] FAISS Documentation, "Facebook AI Similarity Search Library," Available: <https://faiss.ai/>